

Google Most Asked Python DSA Questions

(Covers Arrays, Strings, Linked List, Stack, Queue, Trees, BST, DP & More)

Created by: Abhishek Rathor

Instagram: `SYNTAX ERROR` (`code.abhii07`)

Key Features:

- **75+ Handpicked Interview Questions** from real coding rounds & assessments.
- **Beginner to Advanced Topics:**
 - Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Dynamic Programming → Advanced Backtracking & Trie.
- **Python Implementations** with well-structured **Time & Space Complexities**.
- **Concept Explanations** and **Optimal Approaches** for quick revision.
- **Optimized Code** and patterns used in top product & service-based companies.

Why This Guide?

- ☐ **Save Time:** No need to browse scattered sources — this is your all-in-one prep resource.
- **Practice Smart:** Focused on **most frequently asked DSA problems**
- **Understand Concepts:** Each problem includes logic, patterns, dry-run approach, and edge cases.
- **Placement Ready:** Aligned with Coding Rounds (Online + Technical Interview Levels).

“Success isn’t just about writing code — it’s about solving problems effectively.”

Arrays

1. Two Sum (LeetCode #1)

Problem Statement:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. Assume exactly one solution and no element can be used twice.

Solution Logic:

Use a hashmap to track seen numbers and find complements efficiently.

```
def twoSum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
```

Explanation:

1. Initialize a hashmap `seen` to store numbers and their indices.
2. For each number, calculate the complement (`target - num`).
3. If complement exists in `seen`, return indices.
4. Otherwise, add the current number to `seen`.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

2. Best Time to Buy and Sell Stock (LeetCode #121)

Problem Statement:

Given a list `prices` where `prices[i]` is the stock price on day `i`, find the maximum profit from one buy and one sell.

Solution Logic:

Track minimum price and max profit while traversing once.

```
def maxProfit(prices):
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)
    return max_profit
```

Explanation:

1. Initialize `min_price` to infinity, `max_profit` to zero.
2. Update `min_price` to lowest seen price as we go.
3. Calculate profit for each price and update `max_profit`.
4. Return max profit found.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

3. Merge Intervals (LeetCode #56)

Problem Statement:

Given a list of intervals `[start, end]`, merge all overlapping intervals.

Solution Logic:

Sort intervals and merge overlapping intervals in one pass.

```
def merge(intervals):
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for current in intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]:
            last[1] = max(last[1], current[1])
        else:
            merged.append(current)
    return merged
```

Explanation:

1. Sort intervals by start time.
2. Use `merged` list to store results.
3. Merge if current interval overlaps with last in `merged`.
4. Else, add current interval as new.

Time Complexity: $O(n \log n)$ **Space Complexity:** $O(n)$

4. Product of Array Except Self (LeetCode #238)

Problem Statement:

Given an integer array `nums`, return an array `answer` such that `answer[i]` is the product of all elements except `nums[i]`, without using division.

Solution Logic:

Use prefix and suffix products to compute results in $O(n)$.

```
def productExceptSelf(nums):
    n = len(nums)
    answer = [1] * n
    prefix = 1
    for i in range(n):
        answer[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix
        suffix *= nums[i]
    return answer
```

Explanation:

1. Compute prefix products for each position.
2. Compute suffix products in reverse and multiply.
3. Avoid division by using separate passes.
4. Store result in `answer`.

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (ignoring output array)

5. Move Zeroes (LeetCode #283)

Problem Statement:

Move all zeroes in the array to the end while maintaining the order of non-zero elements.

Solution Logic:

Use two pointers to swap zero and non-zero elements.

```
def moveZeroes(nums):
    j = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[j], nums[i] = nums[i], nums[j]
            j += 1
```

Explanation:

1. Use pointer `j` to track position of next non-zero.
2. Iterate array with `i`.
3. When non-zero found, swap with `j` and increment `j`.
4. Zeroes naturally move to the end.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

6. Contains Duplicate (LeetCode #217)

Problem Statement:

Check if any value appears at least twice in the array.

Solution Logic:

Use a set to track seen elements.

```
def containsDuplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False
```

Explanation:

1. Create empty set `seen`.
2. For each number, check if in `seen`.
3. Return True if duplicate found.
4. Else add number and continue.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

7. Find the Duplicate Number (LeetCode #287)

Problem Statement:

Given an array of $n + 1$ integers where each integer is between 1 and n , find the duplicate number.

Solution Logic:

Use Floyd's Tortoise and Hare (cycle detection) to find duplicate.

```
def findDuplicate(nums):
    slow = fast = nums[0]
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    slow = nums[0]
    while slow != fast:
        slow = nums[slow]
```

```
        fast = nums[fast]
    return slow
```

Explanation:

1. Use two pointers moving at different speeds to detect cycle.
2. Once cycle detected, reset one pointer to start.
3. Move both pointers at same speed until they meet.
4. Meeting point is the duplicate.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

8. Maximum Subarray (Kadane's Algorithm) (LeetCode #53)

Problem Statement:

Find the contiguous subarray with the largest sum.

Solution Logic:

Track current subarray sum and update max sum dynamically.

```
def maxSubArray(nums):
    max_sum = curr_sum = nums[0]
    for num in nums[1:]:
        curr_sum = max(num, curr_sum + num)
        max_sum = max(max_sum, curr_sum)
    return max_sum
```

Explanation:

1. Initialize `curr_sum` and `max_sum` with first element.
2. For each number, decide to start fresh or continue subarray.
3. Update max sum accordingly.
4. Return max sum after iteration.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

9. Set Matrix Zeroes (LeetCode #73)

Problem Statement:

Given a matrix, if an element is 0, set its entire row and column to 0.

Solution Logic:

Use first row and column as markers to track zero rows/columns.

```

def setZeroes(matrix):
    rows, cols = len(matrix), len(matrix[0])
    row_zero = any(matrix[0][j] == 0 for j in range(cols))
    col_zero = any(matrix[i][0] == 0 for i in range(rows))

    for i in range(1, rows):
        for j in range(1, cols):
            if matrix[i][j] == 0:
                matrix[i][0] = 0
                matrix[0][j] = 0

    for i in range(1, rows):
        if matrix[i][0] == 0:
            for j in range(1, cols):
                matrix[i][j] = 0

    for j in range(1, cols):
        if matrix[0][j] == 0:
            for i in range(1, rows):
                matrix[i][j] = 0

    if row_zero:
        for j in range(cols):
            matrix[0][j] = 0

    if col_zero:
        for i in range(rows):
            matrix[i][0] = 0

```

Explanation:

1. Use first row and column to mark zero presence.
2. Mark zeros found in inner matrix on first row/column.
3. Zero out rows and columns based on markers.
4. Handle first row and column separately.

Time Complexity: $O(m*n)$

Space Complexity: $O(1)$

10. Rotate Array (LeetCode #189)

Problem Statement:

Rotate the array to the right by k steps.

Solution Logic:

Reverse whole array, then reverse parts to achieve rotation.

```

def rotate(nums, k):
    k %= len(nums)
    nums.reverse()
    nums[:k] = reversed(nums[k:])
    nums[k:] = reversed(nums[k:])

```

Explanation:

1. Normalize k within array length.
2. Reverse entire array.
3. Reverse first k elements.
4. Reverse remaining elements.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

11. Find All Numbers Disappeared in an Array (LeetCode #448)

Problem Statement:

Find all numbers in the range $[1, n]$ that do not appear in the array.

Solution Logic:

Mark visited numbers by negation and collect missing indices.

```
def findDisappearedNumbers(nums):
    for num in nums:
        index = abs(num) - 1
        if nums[index] > 0:
            nums[index] = -nums[index]
    return [i + 1 for i, num in enumerate(nums) if num > 0]
```

Explanation:

1. For each number, negate the value at corresponding index.
2. Positive values indicate missing numbers.
3. Collect indices + 1 for positive values.
4. Return list of missing numbers.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

12. Top K Frequent Elements (LeetCode #347)

Problem Statement:

Return the k most frequent elements in the array.

Solution Logic:

Use a hashmap for frequency count and a heap to extract top k .

```
from collections import Counter
import heapq
```

```
def topKFrequent(nums, k):  
    count = Counter(nums)  
    return heapq.nlargest(k, count.keys(), key=count.get)
```

Explanation:

1. Count frequency of each element using Counter.
2. Use `heapq.nlargest` to get top k keys by frequency.
3. Return those keys.
4. Efficiently find top k without sorting entire list.

Time Complexity: $O(n \log k)$

Space Complexity: $O(n)$

Strings

13. Longest Substring Without Repeating Characters (LeetCode #3)

Problem Statement:

Given a string, find the length of the longest substring without repeating characters.

Solution Logic:

Use sliding window and a hashmap to track the last seen positions.

```
def lengthOfLongestSubstring(s):  
    char_index = {}  
    left = max_length = 0  
    for right, char in enumerate(s):  
        if char in char_index and char_index[char] >= left:  
            left = char_index[char] + 1  
        char_index[char] = right  
        max_length = max(max_length, right - left + 1)  
    return max_length
```

Explanation:

1. Use `left` and `right` pointers to create a sliding window.
2. Track last seen index of characters in `char_index`.
3. Move `left` when a repeating char is found inside the window.
4. Update max length after expanding `right`.

Time Complexity: $O(n)$

Space Complexity: $O(\min(m, n))$ where m is charset size

14. Valid Anagram (LeetCode #242)

Problem Statement:

Given two strings, determine if one is an anagram of the other.

Solution Logic:

Compare frequency counts of both strings.

```
from collections import Counter

def isAnagram(s, t):
    return Counter(s) == Counter(t)
```

Explanation:

1. Use Counter to get character counts of both strings.
2. Compare the two counters for equality.
3. Return True if identical, False otherwise.
4. Efficient frequency comparison.

Time Complexity: $O(n)$

Space Complexity: $O(1)$ assuming fixed charset

15. Group Anagrams (LeetCode #49)

Problem Statement:

Group a list of strings into anagrams.

Solution Logic:

Sort each string and use it as a hashmap key.

```
from collections import defaultdict

def groupAnagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        groups[tuple(sorted(s))].append(s)
    return list(groups.values())
```

Explanation:

1. Sort each string to get canonical form.
2. Use sorted string tuple as key in hashmap.
3. Append original strings to corresponding groups.
4. Return grouped anagrams.

Time Complexity: $O(n k \log k)$ (k = average string length)

Space Complexity: $O(n k)$

16. Implement strStr() (LeetCode #28)

Problem Statement:

Return the index of the first occurrence of `needle` in `haystack`, or -1 if not found.

Solution Logic:

Use Python's built-in find method or sliding window check.

```
def strStr(haystack, needle):
    if not needle:
        return 0
    for i in range(len(haystack) - len(needle) + 1):
        if haystack[i:i+len(needle)] == needle:
            return i
    return -1
```

Explanation:

1. Handle empty needle case (return 0).
2. Slide over `haystack` checking substrings of length `needle`.
3. Return first matching index or -1 if none.
4. Simple brute force solution.

Time Complexity: $O((n - m) * m)$

Space Complexity: $O(1)$

17. Longest Palindromic Substring (LeetCode #5)

Problem Statement:

Find the longest palindromic substring in a given string.

Solution Logic:

Expand around center for all positions.

```
def longestPalindrome(s):
    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return left + 1, right - 1

    start = end = 0
    for i in range(len(s)):
        l1, r1 = expand(i, i)
        l2, r2 = expand(i, i + 1)
        if r1 - l1 > end - start:
            start, end = l1, r1
        if r2 - l2 > end - start:
```

```
        start, end = l2, r2
    return s[start:end+1]
```

Explanation:

1. Define helper to expand around center and check palindrome.
2. Check odd-length and even-length centers.
3. Update longest palindrome indices.
4. Return substring based on recorded indices.

Time Complexity: $O(n^2)$ **Space Complexity:** $O(1)$

18. Count and Say (LeetCode #38)

Problem Statement:

Given an integer n , generate the n th term of the count-and-say sequence.

Solution Logic:

Iteratively build the string by counting consecutive characters.

```
def countAndSay(n):
    result = "1"
    for _ in range(n - 1):
        temp = ""
        i = 0
        while i < len(result):
            count = 1
            while i + 1 < len(result) and result[i] == result[i + 1]:
                i += 1
                count += 1
            temp += str(count) + result[i]
            i += 1
        result = temp
    return result
```

Explanation:

1. Start with base "1".
2. For each iteration, count consecutive chars and form new string.
3. Use nested loops to process current string.
4. Return final result after n iterations.

Time Complexity: $O(n * 2^n)$ (approximate, due to string growth)**Space Complexity:** $O(2^n)$

19. Valid Parentheses (LeetCode #20)

Problem Statement:

Determine if a string of parentheses is valid (open brackets must be closed in order).

Solution Logic:

Use a stack to match open and close parentheses.

```
def isValid(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for char in s:
        if char in mapping:
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return False
        else:
            stack.append(char)
    return not stack
```

Explanation:

1. Use a stack to track opening brackets.
2. When closing bracket encountered, check stack top matches.
3. Return False on mismatch.
4. Valid if stack empty at the end.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

20. Encode and Decode Strings (LeetCode #271)

Problem Statement:

Design an algorithm to encode a list of strings to a single string and decode it back.

Solution Logic:

Use length-prefixed encoding to avoid delimiter issues.

```
def encode(strs):
    return ''.join(f"{len(s)}#{s}" for s in strs)

def decode(s):
    res, i = [], 0
    while i < len(s):
        j = s.find('#', i)
        length = int(s[i:j])
        res.append(s[j+1:j+1+length])
        i = j + 1 + length
    return res
```

Explanation:

1. Encode each string with its length and a delimiter.
2. During decode, read length, then extract substring.
3. Avoid delimiter collision by length prefix.
4. Reconstruct original list accurately.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

21. Palindrome Permutation (LeetCode #266)

Problem Statement:

Check if any permutation of the string can form a palindrome.

Solution Logic:

A palindrome can have at most one odd-count character.

```
from collections import Counter

def canPermutePalindrome(s):
    counts = Counter(s)
    odd_count = sum(v % 2 for v in counts.values())
    return odd_count <= 1
```

Explanation:

1. Count frequencies of all characters.
2. Sum characters with odd counts.
3. Return True if ≤ 1 , else False.
4. Follows palindrome formation rules.

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (assuming fixed charset)

22. Minimum Window Substring (LeetCode #76)

Problem Statement:

Find the smallest substring in s containing all characters of t .

Solution Logic:

Use sliding window and hashmap to track character requirements.

```
from collections import Counter
```

```

def minWindow(s, t):
    if not s or not t:
        return ""
    dict_t = Counter(t)
    required = len(dict_t)
    l = r = formed = 0
    window_counts = {}
    ans = float('inf'), None, None

    while r < len(s):
        c = s[r]
        window_counts[c] = window_counts.get(c, 0) + 1
        if c in dict_t and window_counts[c] == dict_t[c]:
            formed += 1
        while l <= r and formed == required:
            c = s[l]
            if r - l + 1 < ans[0]:
                ans = (r - l + 1, l, r)
            window_counts[c] -= 1
            if c in dict_t and window_counts[c] < dict_t[c]:
                formed -= 1
            l += 1
        r += 1
    return "" if ans[0] == float('inf') else s[ans[1]:ans[2] + 1]

```

Explanation:

1. Use `dict_t` for target char counts and `window_counts` for current window.
2. Expand right pointer to include characters until all are included.
3. Contract left pointer to minimize window while valid.
4. Track and return minimum window found.

Time Complexity: $O(n)$

Space Complexity: $O(m)$ where m is size of `t`

[Linked Lists](#)

23. Reverse Linked List (LeetCode #206)

Problem Statement:

Reverse a singly linked list.

Solution Logic:

Iteratively reverse pointers by tracking previous and current nodes.

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverseList(head):

```

```

prev = None
curr = head
while curr:
    nxt = curr.next
    curr.next = prev
    prev = curr
    curr = nxt
return prev

```

Explanation:

1. Initialize `prev` as `None`.
2. Traverse list, reverse `next` pointers.
3. Move `prev` and `curr` forward.
4. Return `prev` as new head.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

24. Merge Two Sorted Lists (LeetCode #21)

Problem Statement:

Merge two sorted linked lists and return a sorted merged list.

Solution Logic:

Iteratively pick smaller node from either list.

```

def mergeTwoLists(l1, l2):
    dummy = tail = ListNode()
    while l1 and l2:
        if l1.val < l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next

```

Explanation:

1. Use a dummy node to start merged list.
2. Compare heads of `l1` and `l2`, append smaller.
3. Move pointers accordingly until one list ends.
4. Append remaining nodes and return merged list.

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

25. Linked List Cycle (LeetCode #141)

Problem Statement:

Detect if a linked list has a cycle.

Solution Logic:

Use two pointers moving at different speeds (Floyd's cycle detection).

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Explanation:

1. Initialize slow and fast pointers at head.
2. Move slow by 1 step, fast by 2 steps.
3. If they meet, cycle exists.
4. Return False if fast reaches end.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

26. Remove Nth Node From End (LeetCode #19)

Problem Statement:

Remove the nth node from the end of a linked list.

Solution Logic:

Use two pointers spaced n apart.

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    first = second = dummy
    for _ in range(n + 1):
        first = first.next
    while first:
        first = first.next
        second = second.next
    second.next = second.next.next
    return dummy.next
```

Explanation:

1. Use dummy to handle edge cases.
2. Move `first` pointer `n+1` steps ahead.
3. Move both pointers until `first` reaches end.
4. Remove node after `second`.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

27. Intersection of Two Linked Lists (LeetCode #160)

Problem Statement:

Find the node where two singly linked lists intersect, or `None`.

Solution Logic:

Use two pointers and switch heads to equalize path length.

```
def getIntersectionNode(headA, headB):
    if not headA or not headB:
        return None
    a, b = headA, headB
    while a != b:
        a = a.next if a else headB
        b = b.next if b else headA
    return a
```

Explanation:

1. Use two pointers starting at each head.
2. When pointer reaches end, switch to other list head.
3. Pointers meet at intersection or `None`.
4. Equalizes lengths for sync comparison.

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

28. Palindrome Linked List (LeetCode #234)

Problem Statement:

Check if a singly linked list is a palindrome.

Solution Logic:

Find middle, reverse second half, compare halves.

```
def isPalindrome(head):
    slow = fast = head
    while fast and fast.next:
```

```

        slow = slow.next
        fast = fast.next.next

prev = None
while slow:
    nxt = slow.next
    slow.next = prev
    prev = slow
    slow = nxt

left, right = head, prev
while right:
    if left.val != right.val:
        return False
    left = left.next
    right = right.next
return True

```

Explanation:

1. Use slow and fast pointers to find midpoint.
2. Reverse second half of the list.
3. Compare nodes from start and reversed half.
4. Return True if all match.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

29. Add Two Numbers (LeetCode #2)

Problem Statement:

Add two numbers represented by linked lists where each node contains a digit in reverse order.

Solution Logic:

Iterate nodes, sum digits with carry, create new list.

```

def addTwoNumbers(l1, l2):
    dummy = tail = ListNode()
    carry = 0
    while l1 or l2 or carry:
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0
        carry, out = divmod(val1 + val2 + carry, 10)
        tail.next = ListNode(out)
        tail = tail.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next

```

Explanation:

1. Sum digits and carry as we iterate both lists.
2. Use `divmod` to split sum into digit and carry.
3. Create new node for each digit in result.
4. Continue until both lists and carry are processed.

Time Complexity: $O(\max(n, m))$

Space Complexity: $O(\max(n, m))$

30. Remove Linked List Elements (LeetCode #203)

Problem Statement:

Remove all nodes with a specific value.

Solution Logic:

Use dummy node and iterate, skipping unwanted nodes.

```
def removeElements(head, val):
    dummy = ListNode(next=head)
    current = dummy
    while current.next:
        if current.next.val == val:
            current.next = current.next.next
        else:
            current = current.next
    return dummy.next
```

Explanation:

1. Use dummy node for easy removal of head.
2. Iterate through list, skip nodes with value `val`.
3. Connect previous node to next valid node.
4. Return adjusted list head.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

31. Copy List with Random Pointer (LeetCode #138)

Problem Statement:

Deep copy a linked list with a `random` pointer in addition to `next`.

Solution Logic:

Use hashmap to map original to copied nodes.

```
def copyRandomList(head):
    if not head:
```

```

        return None
    old_to_new = {}
    curr = head
    while curr:
        old_to_new[curr] = ListNode(curr.val)
        curr = curr.next
    curr = head
    while curr:
        old_to_new[curr].next = old_to_new.get(curr.next)
        old_to_new[curr].random = old_to_new.get(curr.random)
        curr = curr.next
    return old_to_new[head]

```

Explanation:

1. Create new nodes and store mapping from old to new.
2. Assign `next` and `random` pointers using mapping.
3. Return copied head.
4. Avoids modifying original list.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

32. Remove Duplicates from Sorted List (LeetCode #83)

Problem Statement:

Delete duplicates from a sorted linked list such that each element appears once.

Solution Logic:

Iterate and remove nodes with duplicate values.

```

def deleteDuplicates(head):
    current = head
    while current and current.next:
        if current.val == current.next.val:
            current.next = current.next.next
        else:
            current = current.next
    return head

```

Explanation:

1. Traverse list with pointer `current`.
2. If `current` and `next` node values are same, skip next node.
3. Otherwise, move to next node.
4. Return cleaned list.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Queues

43. Implement Queue using Stacks (LeetCode #232)

Problem Statement:

Implement a queue using two stacks supporting enqueue, dequeue, peek, and empty operations.

Solution Logic:

Use two stacks: one for enqueue and one for dequeue.

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x):
        self.in_stack.append(x)

    def pop(self):
        self.peek()
        return self.out_stack.pop()

    def peek(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack[-1]

    def empty(self):
        return not self.in_stack and not self.out_stack
```

Explanation:

1. Push elements onto `in_stack`.
2. Pop/peek from `out_stack`.
3. Transfer all elements from `in_stack` to `out_stack` when `out_stack` empty.
4. Check emptiness by both stacks.

Time Complexity: Amortized $O(1)$

Space Complexity: $O(n)$

44. Number of Recent Calls (LeetCode #933)

Problem Statement:

Implement a class to count recent requests in the last 3000 milliseconds.

Solution Logic:

Use a queue to store timestamps and pop outdated ones.

```

from collections import deque

class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t):
        self.q.append(t)
        while self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)

```

Explanation:

1. Append current timestamp.
2. Remove timestamps older than 3000ms from front.
3. Return current size of queue.

Time Complexity: $O(1)$ amortized

Space Complexity: $O(n)$

45. Sliding Window Maximum (LeetCode #239)

Problem Statement:

Find maximum values in each sliding window of size k .

Solution Logic:

Use a deque to store indices of potentially max elements.

```

from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()
    res = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] == i - k:
            dq.popleft()
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res

```

Explanation:

1. Maintain deque with indices in decreasing order of values.
2. Remove smaller elements from back when current is larger.
3. Remove indices out of window from front.
4. Append front value as max for window.

Time Complexity: $O(n)$

Space Complexity: $O(k)$

46. Perfect Squares (LeetCode #279)

Problem Statement:

Find the least number of perfect square numbers that sum to n .

Solution Logic:

Use BFS to explore numbers subtracting squares.

```
from collections import deque
import math

def numSquares(n):
    squares = [i * i for i in range(1, int(math.sqrt(n)) + 1)]
    queue = deque([n])
    level = 0
    visited = set([n])
    while queue:
        level += 1
        for _ in range(len(queue)):
            remainder = queue.popleft()
            for sq in squares:
                next_val = remainder - sq
                if next_val == 0:
                    return level
                if next_val < 0:
                    break
                if next_val not in visited:
                    visited.add(next_val)
                    queue.append(next_val)
    return level
```

Explanation:

1. Generate perfect squares $\leq n$.
2. Use BFS starting from n , subtracting squares.
3. Return level when remainder 0 is found.
4. Avoid revisiting states with `visited`.

Time Complexity: $O(n * \sqrt{n})$

Space Complexity: $O(n)$

47. Implement Circular Queue (LeetCode #622)

Problem Statement:

Design a circular queue supporting enqueue, dequeue, front, rear, and empty operations.

Solution Logic:

Use fixed-size array with pointers.

```
class MyCircularQueue:
    def __init__(self, k):
        self.queue = [0] * k
        self.head = self.tail = -1
        self.size = k

    def enQueue(self, value):
        if self.isFull():
            return False
        if self.isEmpty():
            self.head = 0
        self.tail = (self.tail + 1) % self.size
        self.queue[self.tail] = value
        return True

    def deQueue(self):
        if self.isEmpty():
            return False
        if self.head == self.tail:
            self.head = self.tail = -1
        else:
            self.head = (self.head + 1) % self.size
        return True

    def Front(self):
        return -1 if self.isEmpty() else self.queue[self.head]

    def Rear(self):
        return -1 if self.isEmpty() else self.queue[self.tail]

    def isEmpty(self):
        return self.head == -1

    def isFull(self):
        return (self.tail + 1) % self.size == self.head
```

Explanation:

1. Use array and two pointers for front and rear.
2. Wrap pointers using modulo for circularity.
3. Check full condition when next rear equals front.
4. Handle empty condition when pointers reset.

Time Complexity: $O(1)$ for all ops

Space Complexity: $O(k)$

48. Design Hit Counter (LeetCode #362)

Problem Statement:

Count hits received in the past 5 minutes.

Solution Logic:

Store timestamps in queue and remove outdated ones.

```
from collections import deque

class HitCounter:
    def __init__(self):
        self.hits = deque()

    def hit(self, timestamp):
        self.hits.append(timestamp)

    def getHits(self, timestamp):
        while self.hits and self.hits[0] <= timestamp - 300:
            self.hits.popleft()
        return len(self.hits)
```

Explanation:

1. Record hit timestamps in a queue.
2. Remove hits older than 300 seconds on query.
3. Return number of hits in last 5 minutes.

Time Complexity: $O(1)$ amortized

Space Complexity: $O(n)$

49. Moving Average from Data Stream (LeetCode #346)

Problem Statement:

Calculate moving average from a stream of integers given a window size.

Solution Logic:

Use a queue to store window elements and running sum.

```
from collections import deque

class MovingAverage:
    def __init__(self, size):
        self.size = size
        self.queue = deque()
        self.window_sum = 0

    def next(self, val):
        if len(self.queue) == self.size:
            self.window_sum -= self.queue.popleft()
        self.queue.append(val)
        self.window_sum += val
        return self.window_sum / len(self.queue)
```

Explanation:

1. Keep a fixed-size window using queue.
2. Maintain sum of elements in window.
3. On new input, remove oldest if window full, add new value.
4. Return average by sum divided by current window size.

Time Complexity: $O(1)$ **Space Complexity:** $O(\text{size})$ **50. Number of Islands (BFS) (LeetCode #200)****Problem Statement:**

Count number of islands (connected groups of '1's) in a 2D grid.

Solution Logic:

Use BFS to traverse each island and mark visited.

```
from collections import deque

def numIslands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    visited = set()
    islands = 0

    def bfs(r, c):
        queue = deque([(r, c)])
        visited.add((r, c))
        while queue:
            row, col = queue.popleft()
            for nr, nc in [(row-1,col), (row+1,col), (row,col-1), (row,col+1)]:
                if 0 <= nr < rows and 0 <= nc < cols:
                    if grid[nr][nc] == '1' and (nr,nc) not in visited:
                        visited.add((nr,nc))
                        queue.append((nr,nc))

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                bfs(r, c)
                islands += 1
    return islands
```

Explanation:

1. Iterate all cells, start BFS on unvisited land.
2. BFS visits all connected land cells marking them visited.
3. Increment island count for each BFS.
4. Return total islands.

Time Complexity: $O(\text{rows} * \text{cols})$

Space Complexity: $O(\text{rows} * \text{cols})$

51. Shortest Path in Binary Matrix (LeetCode #1091)

Problem Statement:

Find shortest clear path from top-left to bottom-right in a binary matrix moving 8 directions.

Solution Logic:

Use BFS to find shortest path length.

```
from collections import deque

def shortestPathBinaryMatrix(grid):
    n = len(grid)
    if grid[0][0] or grid[n-1][n-1]:
        return -1
    directions = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
    queue = deque([(0,0,1)])
    visited = set([(0,0)])

    while queue:
        r, c, dist = queue.popleft()
        if r == c == n-1:
            return dist
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0 and (nr,nc) not in visited:
                visited.add((nr,nc))
                queue.append((nr,nc,dist+1))

    return -1
```

Explanation:

1. Check start and end are clear.
2. Use BFS with distance tracking.
3. Visit neighbors in 8 directions if unblocked.
4. Return distance when destination reached.

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

52. Number of Islands II (LeetCode #305)

Problem Statement:

Given a list of positions to add land in a grid, return number of islands after each addition.

Solution Logic:

Use Union-Find data structure to merge islands dynamically.

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = 0

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rootX, rootY = self.find(x), self.find(y)
        if rootX != rootY:
            if self.rank[rootX] > self.rank[rootY]:
                self.parent[rootY] = rootX
            elif self.rank[rootX] < self.rank[rootY]:
                self.parent[rootX] = rootY
            else:
                self.parent[rootY] = rootX
                self.rank[rootX] += 1
            self.count -= 1

def numIslands2(m, n, positions):
    uf = UnionFind(m * n)
    grid = [[0]*n for _ in range(m)]
    res = []
    for r, c in positions:
        if grid[r][c] == 1:
            res.append(uf.count)
            continue
        grid[r][c] = 1
        uf.count += 1
        idx = r * n + c
        for nr, nc in [(r-1,c), (r+1,c), (r,c-1), (r,c+1)]:
            if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 1:
                uf.union(idx, nr * n + nc)
        res.append(uf.count)
    return res
```

Explanation:

1. Initialize Union-Find for all cells.
2. For each position, add land and increment island count.
3. Union adjacent lands to merge islands.
4. Append current island count after each addition.

Time Complexity: $O(k \alpha(mn))$ (k = number of positions, α = inverse Ackermann)

Space Complexity: $O(mn)$

Trees and Binary Search Trees

53. Validate Binary Search Tree (LeetCode #98)

Problem Statement:

Check if a binary tree is a valid BST ($\text{left} < \text{root} < \text{right}$).

Solution Logic:

Use in-order traversal to check if values are strictly increasing.

```
def isValidBST(root):
    prev = [float('-inf')]

    def inorder(node):
        if not node:
            return True
        if not inorder(node.left):
            return False
        if node.val <= prev[0]:
            return False
        prev[0] = node.val
        return inorder(node.right)

    return inorder(root)
```

Explanation:

1. Traverse in-order to get nodes in sorted order.
2. Check if current node value > previous value.
3. If violated, return False.
4. Return True if all nodes satisfy BST property.

Time Complexity: $O(n)$

Space Complexity: $O(h)$ — height of tree (stack space)

54. Lowest Common Ancestor of BST (LeetCode #235)

Problem Statement:

Find the lowest common ancestor (LCA) of two nodes in a BST.

Solution Logic:

Utilize BST properties to move towards LCA.

```
def lowestCommonAncestor(root, p, q):
    while root:
        if p.val < root.val > q.val:
            root = root.left
        elif p.val > root.val < q.val:
```

```
        root = root.right
    else:
        return root
```

Explanation:

1. If both p and q less than root, go left.
2. If both greater, go right.
3. Otherwise, root is LCA.

Time Complexity: $O(h)$ **Space Complexity:** $O(1)$

55. Binary Tree Level Order Traversal (LeetCode #102)

Problem Statement:

Return level order traversal of binary tree nodes' values.

Solution Logic:

Use BFS with queue to traverse each level.

```
from collections import deque

def levelOrder(root):
    if not root:
        return []
    res, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        res.append(level)
    return res
```

Explanation:

1. Use queue to hold nodes at current level.
2. Traverse all nodes at level, enqueue children.
3. Collect values level-wise.

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$

56. Serialize and Deserialize Binary Tree (LeetCode #297)

Problem Statement:

Design an algorithm to serialize and deserialize a binary tree.

Solution Logic:

Use preorder traversal with markers for null nodes.

```
def serialize(root):
    vals = []
    def preorder(node):
        if node:
            vals.append(str(node.val))
            preorder(node.left)
            preorder(node.right)
        else:
            vals.append('#')
    preorder(root)
    return ','.join(vals)

def deserialize(data):
    vals = iter(data.split(','))
    def build():
        val = next(vals)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = build()
        node.right = build()
        return node
    return build()
```

Explanation:

1. Serialize with preorder adding '#' for nulls.
2. Deserialize recursively by reading values.
3. Construct nodes and their children accordingly.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

57. Binary Tree Maximum Path Sum (LeetCode #124)

Problem Statement:

Find the maximum path sum in a binary tree, where a path can start and end at any node.

Solution Logic:

Use DFS to find max path sum including current node.

```
def maxPathSum(root):
    max_sum = float('-inf')
```

```
def dfs(node):
    nonlocal max_sum
    if not node:
        return 0
    left = max(dfs(node.left), 0)
    right = max(dfs(node.right), 0)
    max_sum = max(max_sum, node.val + left + right)
    return node.val + max(left, right)

dfs(root)
return max_sum
```

Explanation:

1. DFS returns max gain from left/right subtree ignoring negatives.
2. Update global max with sum through current node.
3. Return max path sum including current node plus one branch.

Time Complexity: $O(n)$

Space Complexity: $O(h)$

58. Invert Binary Tree (LeetCode #226)

Problem Statement:

Invert a binary tree (mirror it).

Solution Logic:

Recursively swap left and right children.

```
def invertTree(root):
    if not root:
        return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

Explanation:

1. Recursively invert left and right subtrees.
2. Swap left and right pointers.
3. Return inverted tree.

Time Complexity: $O(n)$

Space Complexity: $O(h)$

59. Construct Binary Tree from Preorder and Inorder Traversal (LeetCode #105)

Problem Statement:

Build binary tree from preorder and inorder traversal arrays.

Solution Logic:

Use preorder root and inorder partition to build tree recursively.

```
def buildTree(preorder, inorder):
    if not preorder or not inorder:
        return None
    root_val = preorder[0]
    root = TreeNode(root_val)
    mid = inorder.index(root_val)
    root.left = buildTree(preorder[1:mid+1], inorder[:mid])
    root.right = buildTree(preorder[mid+1:], inorder[mid+1:])
    return root
```

Explanation:

1. First preorder element is root.
2. Find root in inorder to split left/right subtrees.
3. Recursively build left and right subtrees.

Time Complexity: $O(n^2)$ — can be improved with hashmap

Space Complexity: $O(n)$

60. Symmetric Tree (LeetCode #101)

Problem Statement:

Check if a binary tree is symmetric (mirror of itself).

Solution Logic:

Use recursion to compare left and right subtrees.

```
def isSymmetric(root):
    def isMirror(t1, t2):
        if not t1 and not t2:
            return True
        if not t1 or not t2:
            return False
        return (t1.val == t2.val) and isMirror(t1.left, t2.right) and isMirror(t1.right, t2.left)
    return isMirror(root, root)
```

Explanation:

1. Check if two subtrees are mirror images.
2. Compare root values and recursive children crosswise.
3. Return True if all match.

Time Complexity: $O(n)$

Space Complexity: $O(h)$

61. Kth Smallest Element in a BST (LeetCode #230)

Problem Statement:

Find the kth smallest element in a BST.

Solution Logic:

Inorder traversal gives sorted order; return kth element.

```
def kthSmallest(root, k):
    stack = []
    while True:
        while root:
            stack.append(root)
            root = root.left
        root = stack.pop()
        k -= 1
        if k == 0:
            return root.val
        root = root.right
```

Explanation:

1. Use iterative inorder traversal.
2. Decrement k on each visited node.
3. Return value when k reaches 0.

Time Complexity: $O(h + k)$

Space Complexity: $O(h)$

62. Insert into a Binary Search Tree (LeetCode #701)

Problem Statement:

Insert a value into a BST and return the root.

Solution Logic:

Recursively insert in correct subtree.

```
def insertIntoBST(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insertIntoBST(root.left, val)
    else:
        root.right = insertIntoBST(root.right, val)
```

```
return root
```

Explanation:

1. If root is None, insert here.
2. If $val < root$, insert left.
3. Else insert right.
4. Return updated root.

Time Complexity: $O(h)$

Space Complexity: $O(h)$

Graphs

63. Number of Islands (DFS) (LeetCode #200)

Problem Statement:

Count the number of islands (connected groups of '1's) in a 2D grid using DFS.

Solution Logic:

Perform DFS to mark connected land as visited.

```
def numIslands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    visited = set()
    count = 0

    def dfs(r, c):
        if (r < 0 or r >= rows or c < 0 or c >= cols or
            grid[r][c] == '0' or (r, c) in visited):
            return
        visited.add((r, c))
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                dfs(r, c)
                count += 1
    return count
```

Explanation:

1. Traverse grid and call DFS on unvisited land.
2. DFS visits and marks all connected land cells.

3. Increment island count per DFS call.
4. Return total islands.

Time Complexity: $O(\text{rows} * \text{cols})$

Space Complexity: $O(\text{rows} * \text{cols})$

64. Clone Graph (LeetCode #133)

Problem Statement:

Clone an undirected graph represented as adjacency list.

Solution Logic:

Use DFS with hashmap to copy nodes and neighbors.

```
def cloneGraph(node):
    if not node:
        return None
    visited = {}

    def dfs(n):
        if n in visited:
            return visited[n]
        copy = Node(n.val)
        visited[n] = copy
        for nei in n.neighbors:
            copy.neighbors.append(dfs(nei))
        return copy

    return dfs(node)
```

Explanation:

1. Use DFS to visit each node.
2. Store clones in hashmap to avoid duplicates.
3. Recursively clone neighbors.
4. Return cloned graph root.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

65. Course Schedule (LeetCode #207)

Problem Statement:

Given prerequisites, check if you can finish all courses (detect cycle in directed graph).

Solution Logic:

Use DFS to detect cycles.

```
def canFinish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    for u, v in prerequisites:
        graph[v].append(u)
    visited = [0] * numCourses

    def dfs(i):
        if visited[i] == 1:
            return False
        if visited[i] == 2:
            return True
        visited[i] = 1
        for neighbor in graph[i]:
            if not dfs(neighbor):
                return False
        visited[i] = 2
        return True

    return all(dfs(i) for i in range(numCourses))
```

Explanation:

1. Build adjacency list from prerequisites.
2. Use DFS with states: 0=unvisited, 1=visiting, 2=visited.
3. Detect back edge (cycle) if node visited twice in stack.
4. Return False if cycle found.

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

66. Course Schedule II (LeetCode #210)**Problem Statement:**

Return an order of courses you should take to finish all courses (topological sort).

Solution Logic:

Use DFS or BFS to perform topological sorting.

```
from collections import deque

def findOrder(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses
    for u, v in prerequisites:
        graph[v].append(u)
        indegree[u] += 1

    queue = deque([i for i in range(numCourses) if indegree[i] == 0])
    order = []
```

```

while queue:
    node = queue.popleft()
    order.append(node)
    for neighbor in graph[node]:
        indegree[neighbor] -= 1
        if indegree[neighbor] == 0:
            queue.append(neighbor)

return order if len(order) == numCourses else []

```

Explanation:

1. Build graph and indegree array.
2. Enqueue all courses with no prerequisites.
3. Remove edges and enqueue next courses.
4. Return order if all courses included.

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

67. Pacific Atlantic Water Flow (LeetCode #417)

Problem Statement:

Find cells where water can flow to both the Pacific and Atlantic oceans in a height matrix.

Solution Logic:

Use DFS/BFS from oceans inward and find intersecting cells.

```

def pacificAtlantic(heights):
    if not heights:
        return []
    rows, cols = len(heights), len(heights[0])

    def dfs(r, c, visited, prev_height):
        if (r < 0 or r >= rows or c < 0 or c >= cols or
            (r, c) in visited or heights[r][c] < prev_height):
            return
        visited.add((r, c))
        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
            dfs(r+dr, c+dc, visited, heights[r][c])

    pacific, atlantic = set(), set()
    for i in range(rows):
        dfs(i, 0, pacific, heights[i][0])
        dfs(i, cols - 1, atlantic, heights[i][cols - 1])
    for j in range(cols):
        dfs(0, j, pacific, heights[0][j])
        dfs(rows - 1, j, atlantic, heights[rows - 1][j])

    return list(pacific & atlantic)

```

Explanation:

1. DFS from ocean borders inward if next cell height $\geq$ current.
2. Track reachable cells from both oceans.
3. Intersection gives cells that flow to both.

Time Complexity: $O(\text{rows} * \text{cols})$ **Space Complexity:** $O(\text{rows} * \text{cols})$

68. Number of Connected Components in an Undirected Graph (LeetCode #323)**Problem Statement:**

Find the number of connected components in an undirected graph.

Solution Logic:

Use DFS or Union-Find to count components.

```
def countComponents(n, edges):
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited = set()
    count = 0

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)

    for i in range(n):
        if i not in visited:
            dfs(i)
            count += 1
    return count
```

Explanation:

1. Build adjacency list.
2. DFS on unvisited nodes to mark component.
3. Increment count per DFS call.
4. Return total connected components.

Time Complexity: $O(n + e)$ **Space Complexity:** $O(n + e)$

69. Graph Valid Tree (LeetCode #261)

Problem Statement:

Check if given edges make a valid tree with n nodes.

Solution Logic:

Check if graph is connected and edges = $n-1$.

```
def validTree(n, edges):
    if len(edges) != n - 1:
        return False
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited = set()

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)

    dfs(0)
    return len(visited) == n
```

Explanation:

1. Tree must have exactly $n-1$ edges.
2. Graph must be fully connected.
3. DFS to check connectivity.
4. Return True if connected and edges correct.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

70. Word Ladder (LeetCode #127)

Problem Statement:

Find shortest transformation sequence length from `beginWord` to `endWord` changing one letter at a time.

Solution Logic:

Use BFS on word graph where edges represent single letter difference.

```
from collections import deque

def ladderLength(beginWord, endWord, wordList):
    wordSet = set(wordList)
    if endWord not in wordSet:
        return 0
```

```

queue = deque([(beginWord, 1)])

while queue:
    word, length = queue.popleft()
    if word == endWord:
        return length
    for i in range(len(word)):
        for c in 'abcdefghijklmnopqrstuvwxyz':
            new_word = word[:i] + c + word[i+1:]
            if new_word in wordSet:
                wordSet.remove(new_word)
                queue.append((new_word, length + 1))

return 0

```

Explanation:

1. BFS from `beginWord`.
2. Generate all single letter transformations.
3. Enqueue valid words and remove to avoid repeats.
4. Return length when `endWord` found.

Time Complexity: $O(MN26)$ (M =word length, N =word list size)

Space Complexity: $O(N)$

71. Minimum Height Trees (LeetCode #310)

Problem Statement:

Find roots of minimum height trees in an undirected graph.

Solution Logic:

Remove leaves iteratively until 1 or 2 nodes remain.

```

from collections import deque

def findMinHeightTrees(n, edges):
    if n == 1:
        return [0]
    graph = [[] for _ in range(n)]
    degree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
        degree[u] += 1
        degree[v] += 1

    leaves = deque([i for i in range(n) if degree[i] == 1])
    remaining = n

    while remaining > 2:
        leaves_count = len(leaves)
        remaining -= leaves_count
        for _ in range(leaves_count):

```

```

        leaf = leaves.popleft()
        for neighbor in graph[leaf]:
            degree[neighbor] -= 1
            if degree[neighbor] == 1:
                leaves.append(neighbor)
    return list(leaves)

```

Explanation:

1. Initialize leaves (nodes with degree 1).
2. Remove leaves layer by layer.
3. Update degrees of neighbors.
4. Remaining nodes are MHT roots.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

72. Reconstruct Itinerary (LeetCode #332)

Problem Statement:

Given airline tickets, reconstruct itinerary in lexical order starting from 'JFK'.

Solution Logic:

Hierholzer's algorithm for Eulerian path using DFS.

```

from collections import defaultdict

def findItinerary(tickets):
    graph = defaultdict(list)
    for src, dst in sorted(tickets, reverse=True):
        graph[src].append(dst)
    route = []

    def visit(airport):
        while graph[airport]:
            visit(graph[airport].pop())
        route.append(airport)

    visit('JFK')
    return route[::-1]
]

```

Explanation:

1. Build graph with adjacency sorted in reverse lexical order.
2. DFS visit consuming edges to build path.
3. Append airport when no further path.
4. Reverse route to get itinerary.

Time Complexity: $O(E \log E)$ due to sorting edges

Space Complexity: $O(E)$

Dynamic Programming / Backtracking

73. Climbing Stairs (LeetCode #70)

Problem Statement:

You can climb 1 or 2 steps at a time. How many distinct ways to reach the top (n steps)?

Solution Logic:

Use DP: ways to reach step $n = \text{ways}(n-1) + \text{ways}(n-2)$.

```
def climbStairs(n):
    if n <= 2:
        return n
    dp = [0]*(n+1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

Explanation:

1. Base cases: 1 way for step 1, 2 ways for step 2.
2. For each step, sum ways from previous two steps.
3. Return ways for nth step.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

74. Word Break (LeetCode #139)

Problem Statement:

Given a string and word dictionary, determine if the string can be segmented into dictionary words.

Solution Logic:

Use DP to check substrings if they can be segmented.

```
def wordBreak(s, wordDict):
    wordSet = set(wordDict)
    dp = [False]*(len(s)+1)
    dp[0] = True
    for i in range(1, len(s)+1):
        for j in range(i):
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True
```

```
        break
    return dp[-1]
```

Explanation:

1. $dp[i]$ is True if $s[:i]$ can be segmented.
2. Check all substrings $s[j:i]$.
3. If $s[j:i]$ in dictionary and $dp[j]$ True, $dp[i]$ True.
4. Return dp for full string.

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

75. Combination Sum (LeetCode #39)

Problem Statement:

Find all unique combinations where candidate numbers sum to target. Candidates can be used unlimited times.

Solution Logic:

Use backtracking to explore combinations.

```
def combinationSum(candidates, target):
    res = []
    candidates.sort()

    def backtrack(start, path, target):
        if target == 0:
            res.append(path[:])
            return
        if target < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, target - candidates[i])
            path.pop()

    backtrack(0, [], target)
    return res
```

Explanation:

1. Sort candidates for pruning.
2. Backtrack: add candidates to path if $target \geq 0$.
3. Append path when $target == 0$.
4. Explore next candidates from current index.

Time Complexity: $O(2^t)$ ($t = target$)

Space Complexity: $O(t)$

6 focused, high-impact tips to help you crack Google SDE interviews.

1) Master fundamentals & problem patterns

Why: Google interviews test core DS + alg patterns (sliding window, two pointers, DFS/BFS, heaps, union-find, DP, graphs).

Do this: Drill 50–150 problems across arrays/strings/trees/graphs/DP. For each problem, identify the pattern, then solve 3 variations of that pattern. Track patterns in a notebook (or doc).

2) Practice clear, verbal problem-solving

Why: Interviewers assess how you think — not just the final code. Clear communication wins.

Do this: In mock interviews, always: restate the problem, ask clarifying Qs, outline brute force → optimize, pick an approach, write pseudocode, then code.

3) Simulate real interviews (timed + whiteboard style)

Why: Time pressure and no-autocomplete environments change how you code.

Do this: Do 45–60 minute mock sessions: 1 medium + 1 easy/medium, on a blank editor or whiteboard. Practice coding by hand or in a plain text editor with no linting. Review and improve after each mock.

4) Write clean, correct, well-tested code

Why: Google expects readable, bug-free code and good testing/thinking about edge cases.

Do this: Use meaningful variable names, handle edge cases, and run 3–5 test cases aloud (including boundaries). Always state complexity (time & space) at the end.

5) Prepare system design + role-appropriate topics

Why: For SDE II & above (and sometimes for SDE I), system design matters; architecture shows product & engineering judgment.

Do this: Learn design patterns (caching, load balancing, DB choices), practice 3–4 end-to-end designs (e.g., URL shortener, news feed, rate limiter). Be ready to discuss trade-offs, scaling, and APIs.

6) Behavioral & hiring process prep

Why: Google evaluates “Googliness” (collaboration, leadership, problem ownership) alongside technical skills. Good interview signals make a big difference.

Do this: Prepare 6–8 STAR stories covering impact, conflict, ambiguity, learning from failure, technical leadership. Also: polish your resume to highlight impact (metrics), get referrals if possible, and be ready to discuss any project on your resume deeply.

Quick daily plan (practical)

- **60–90 min/day:** Problem solving (mix of new + review)
- **30 min/day:** Read/write notes on patterns or revisit mistakes
- **Weekly:** 1 mock (timed), 1 system-design walkthrough, 1 behavioral rehearsal

Congratulations! You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills. Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: [@SYNTAXERROR](#). Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved. This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.